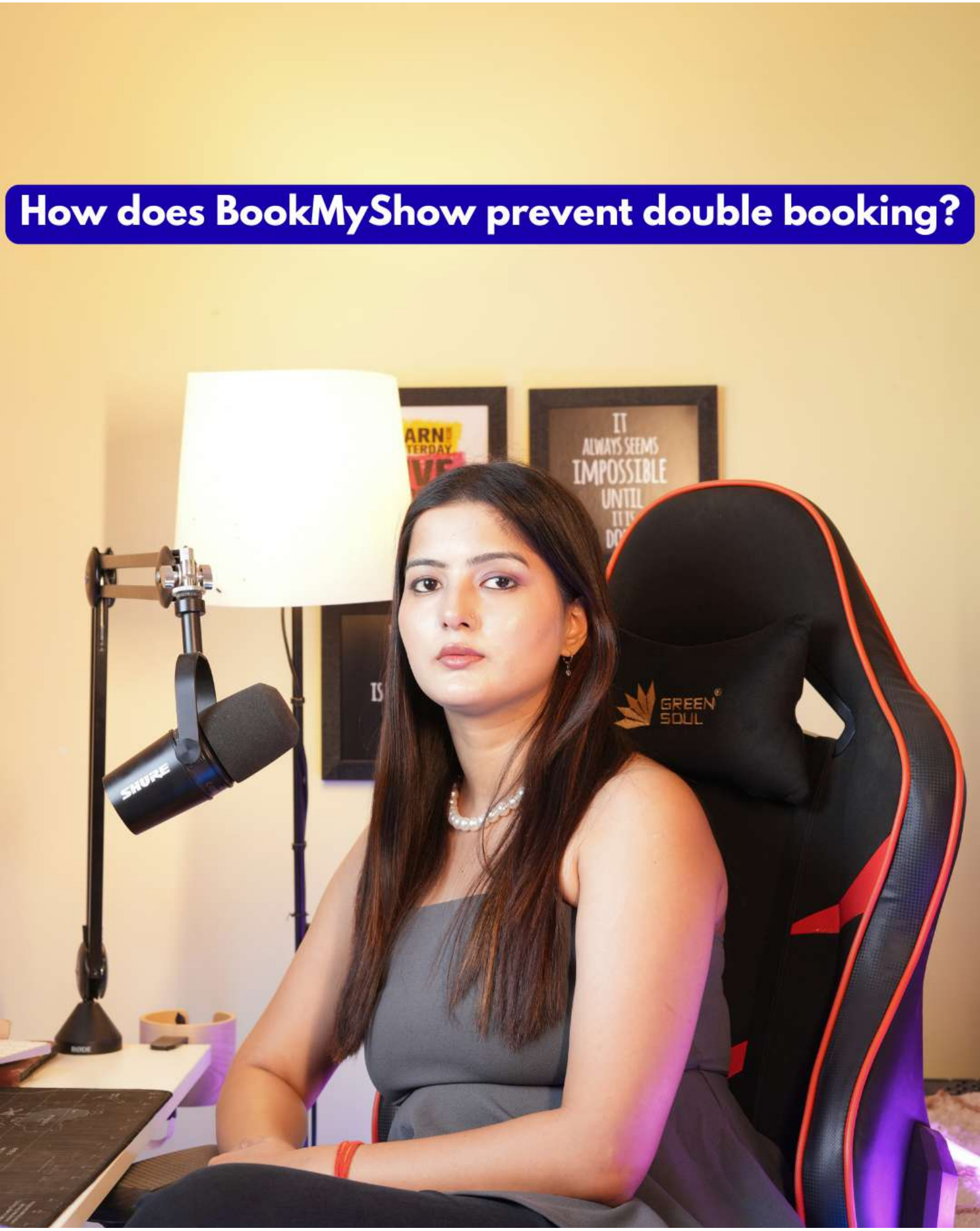
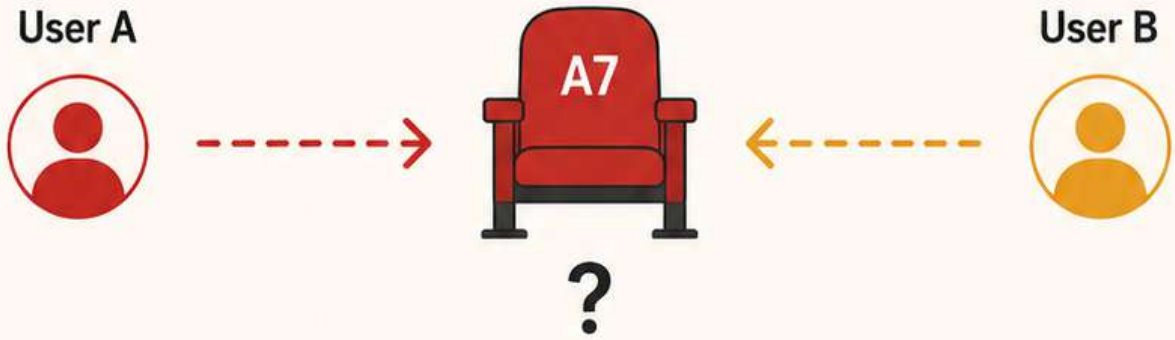
How does BookMyShow prevent double booking?



BOOKMYSHOW
THE PROBLEM

Imagine it's the first day of a blockbuster movie.
Only **1 seat** is left.

Two users click "Book Now"
at exactly the same time.



Both users expect:

- Instant booking
- Successful payment
- Reserved seat

But there's a problem...

Only **one person** can own **Seat A7.**
So how does BookMyShow decide who gets it?

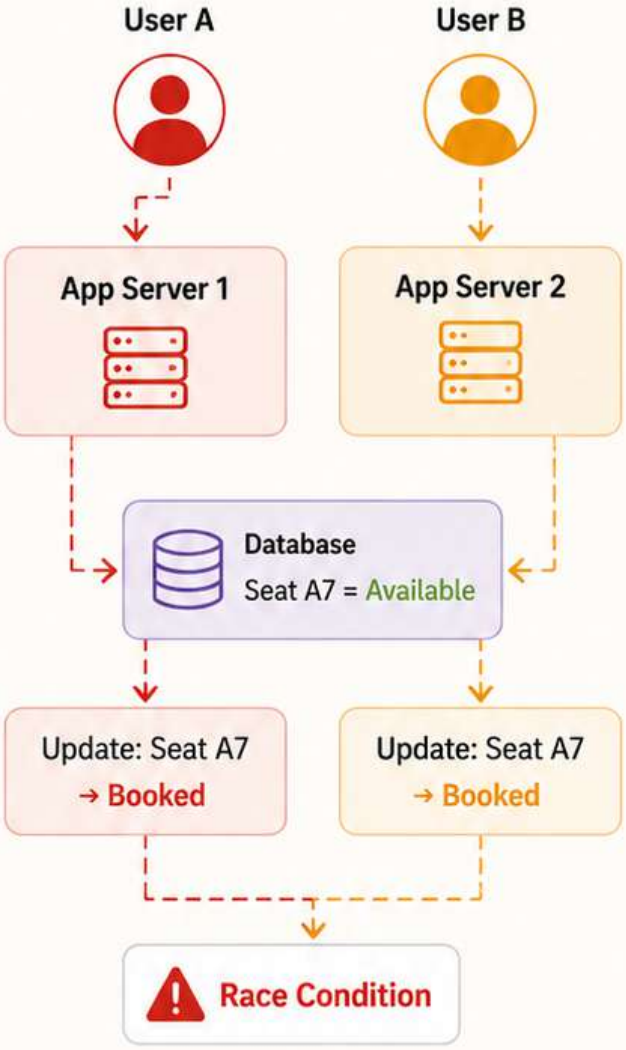
If both requests succeed, the same seat gets sold twice.

This is called a **Race Condition.**

BOOKMYSHOW
WHY THIS HAPPENS

The system processes **both requests** at the same time.

1. **User A and User B** send booking requests at the same time.
2. **Both requests hit** different application servers.
3. **Both servers check** the database. Seat A7 = Available
4. **Both servers try to** reserve Seat A7 simultaneously.
5. **Both updates succeed.** The same seat gets **booked twice!**



Key Takeaway

Checking availability and booking are two separate steps.
When done together by multiple users,
it can lead to **inconsistent results.**

BOOKMYSHOW

WHY SIMPLE DATABASE UPDATES AREN'T ENOUGH

Simple read → update operations break under **high concurrency**.

WHAT HAPPENS?

1. Read
Both servers read Seat A7 = **Available**

2. Process
Both think the seat is still free.

3. Update
Both update the seat to **Booked**.

4. Result
Both updates succeed. Seat is **sold twice.**

WHY THIS HAPPENS

No isolation
Both transactions read the same data before either writes.

No atomicity
Check (is seat free) and update (book seat) are separate steps.

High concurrency
Millions of users trying to book at the same time.

Proper concurrency control is required
Using transactions, locks, or optimistic concurrency.

KEY TAKEAWAY

The availability check and booking must happen **atomically** using proper concurrency control.

This ensures only one user can book the seat.

BOOKMYSHOW
HOW DO WE PREVENT DOUBLE BOOKING?

BookMyShow uses multiple strategies to make sure only one user can book a seat.

CORE STRATEGIES

Lock the Seat
Temporarily lock the seat when a user starts booking.

Atomic Update
Check availability and book the seat in one atomic step.

Lock Timeout
Release the lock if payment is not completed.

Idempotency
Ensure repeated requests don't book again.

Strong Consistency
Use consistent storage or locks to avoid conflicts.

High Concurrency
Systems designed to handle millions of simultaneous users.

HOW IT WORKS (HIGH LEVEL FLOW)

1. **User selects seat** — Seat A7 is locked for the user.
2. **System checks availability** — Re-checks if the seat is still available.
3. **Atomic booking** — Seat is booked in an atomic transaction.
4. **Payment** — User completes the payment.
5. **Confirmed** — Booking is confirmed and seat is marked as booked.
6. **Release / Expire** — If payment fails or expires, lock is released.

KEY TAKEAWAY

The key is to make the check, lock, and book process atomic so that only one user can successfully book the seat.

BOOKMYSHOW

LOCKING MECHANISMS WE CAN USE

Different ways to lock a seat and prevent double booking.

1. PESSIMISTIC LOCKING

Lock the seat in the database before booking.

How it works

SELECT seat
FOR UPDATE;

Blocks other transactions.

Best for

High consistency scenarios.

2. OPTIMISTIC LOCKING

Use version number to detect conflicts.

How it works

Update seat
WHERE version = X;

If no rows updated, someone else booked it.

Best for

High read, low write scenarios.

3. DISTRIBUTED LOCK

Use Redis/Zookeeper to acquire a lock.

How it works

SET lock:seat:A7 NX
EX 10

Only one request gets the lock.

Best for

Distributed systems with many servers.

4. QUEUE BASED BOOKING

Requests are processed one by one.

How it works

Add to queue
→ Process sequentially

No two requests process at the same time.

Best for

Extreme high concurrency scenarios.

COMPARISON

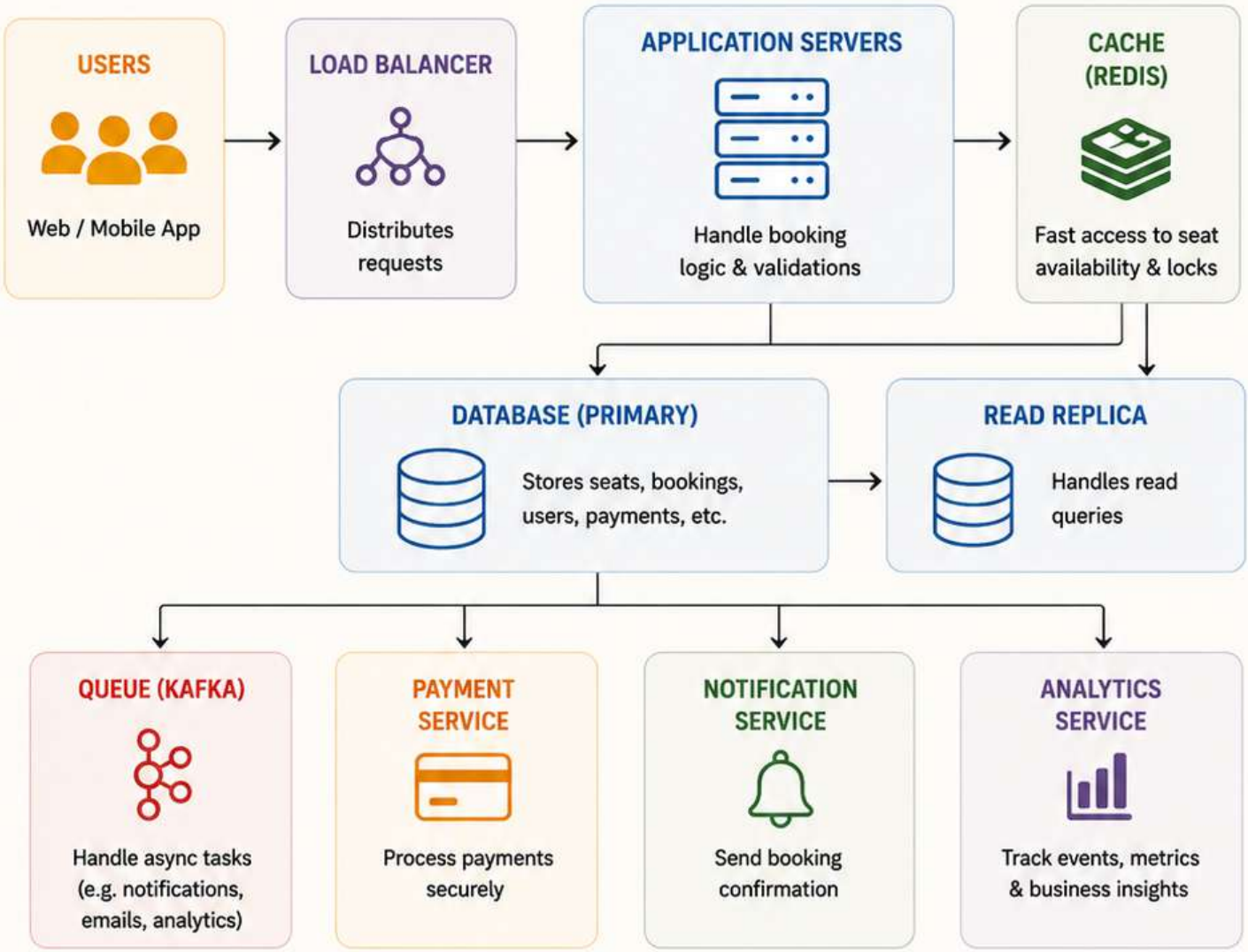
Approach	Consistency	Performance	Complexity	Use Case
Pessimistic Locking	Very High	Lower	Medium	Important bookings, low concurrency
Optimistic Locking	High	High	Low	High read, low write
Distributed Lock	Very High	High	Medium	Multiple servers, real-time locking
Queue Based	Very High	Lower	High	Extreme high traffic

KEY TAKEAWAY

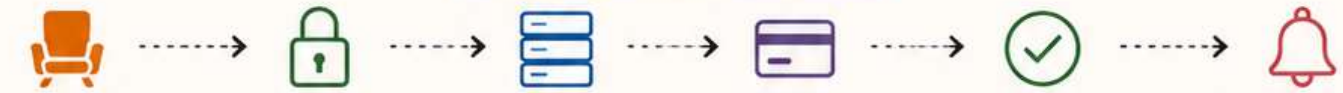
There is no one-size-fits-all solution.
Choose the right locking mechanism based on your traffic, consistency needs and system architecture.

BOOKMYSHOW
HIGH LEVEL SYSTEM DESIGN

Let's see how BookMyShow's booking system works at a high level.



HOW IT WORKS (IN SHORT)



KEY TAKEAWAY

The system is designed for high concurrency, low latency and strong consistency using a combination of caching, locking, databases and async processing.

BOOKMYSHOW
SCALING FOR MILLIONS

The system is built to handle **massive traffic** and **high concurrency**.

USERS
Millions of users from web / mobile app

DNS & CDN
Route users to nearest edge location

LOAD BALANCER
Distribute traffic across multiple servers

APPLICATION SERVERS
Stateless servers handle booking requests

CACHE LAYER (REDIS)
Store seat availability, locks, sessions, and metadata

DATA LAYER

PRIMARY DATABASE
Stores critical data (seats, bookings, users, payments)

READ REPLICAS
Handle read-heavy queries and reporting

SHARDING
Distribute data across multiple shards (shows, cities, users)

QUEUE SERVICE
Handle async tasks (e.g. notifications, emails, analytics, reports)

PAYMENT SERVICE
Process payments securely and asynchronously

NOTIFICATION SERVICE
Send booking confirmations via email / SMS / push

ANALYTICS SERVICE
Track events, metrics, trends and business insights

SCALABILITY FEATURES

Horizontal Scaling
Add more servers to handle traffic spikes

Auto Scaling
Automatically scale servers based on demand

Multi-Region
Deploy in multiple regions for low latency & high availability

High Availability
Redundancy at every layer to avoid downtime

Monitoring & Alerts
Real-time monitoring to detect and fix issues fast

KEY TAKEAWAY

BookMyShow's system is designed to scale horizontally, handle high concurrency, and ensure a seamless booking experience even during peak demand.

BOOKMYSHOW
CONCLUSION

Preventing double booking at scale is a complex challenge,
but with the **right strategies** and **system design**, it's absolutely possible.

THE CHALLENGE

Handling millions of users trying to book seats at the same time without allowing double booking.

OUR APPROACH

Use locking mechanisms, caching, queues, databases, and microservices working together.

THE RESULT

A highly scalable, reliable and fast system that ensures a seamless booking experience for millions of users.

THE KEY LESSON

- Think beyond the database
- Design for concurrency from day one
- Build systems that are scalable, fault-tolerant and user-focused

REMEMBER

It's not just about writing code,
it's about designing systems that people can trust.

Loved this breakdown? Don't forget to


Save


Share


Comment


Follow

Follow **iamsaumyaawasthi** for more such breakdowns ❤